

- **or/and** 返回 x,y 本身 not 返回 True/False x,y 可以是任何值: integer boolean 表达式 string
- `print("Good morning! ", name, "!", sep="")` 默认情况 `print()` 会在多个参数之间添加空格 (`sep=""`)
- **f-string** 以 f 开头, 通过 {变量名} 直接在字符串中嵌入变量 (无需手动拼接)
- **Raw string** \ 作为普通字符不转义 结尾不能是单个 \ 需用 \\ 或多字符结尾
- 左对齐: <9.2f (左对齐, 总宽 9, 2 位小数) 右对齐: >9.2f (右对齐, 总宽 9, 2 位小数)
- :^9.2f (居中对齐, 总宽 9, 2 位小数) {3.14159265359:09.3f} 00003.142 {num :\*<9.2f}|")
- 使用下划线 \_ 作为循环变量, 暗示循环体中不使用迭代器提供的具体数值
- 要「索引 + 元素」→ 用 **enumerate**; 要「多序列并行配对」→ 用 **zip**:

`def special_print(name, *args, **kwargs):` These parameters are passed on to print()  
`print(f"({name})")`  
`print(*args, **kwargs)`  
`special_print("Gibson", "Good morning!")` → [Gibson]  
 Good morning!  
`numbers = map(lambda x: x**2, range(1, 11))`

- Boolean/list Index 返回数组的 **copy** Transposition (a.T) 返回数组的 **view**
- 普通切片 a[:, 0:2, 0:3]) 各轴独立取所有组合; 高级索引 (列表做索引) 各轴的列表索引会元素级配对, 取配对组合
- 列表索引长度须一致, 除非仅有一个为 1 可广播否则报错 `result = a[:, [0], [0, 1, 2]] → (0,0), (0,1), (0,2)`
- 广播机制: ① 维度相同或低维的左边填充 1 ② 每个维度长度相同或一个为 1 可扩展
- Array 的 **Boolean Mask** 和 **Indexing** `mask = a >= 3 / np.array([[true, false], ...])` `print(a[mask])`

**print(a.sum(axis=1))**

| Method                   | Description           |
|--------------------------|-----------------------|
| <code>np.sum()</code>    | Total of all elements |
| <code>np.mean()</code>   | Average value         |
| <code>np.std()</code>    | Standard deviation    |
| <code>np.var()</code>    | Variance              |
| <code>np.min()</code>    | Minimum value         |
| <code>np.max()</code>    | Maximum value         |
| <code>np.median()</code> | Median value          |

`np.vstack()`  
 • Combine arrays by stacking rows together  
`np.hstack()`  
 • Combine arrays by stacking columns together  
`np.dstack()`  
 • Combine arrays by stacking elements in the last axis together

**Examples of Stacking Arrays – dstack**

Given these arrays:

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
```

Using `np.dstack()`:

```
print(np.dstack((a, b)))
```

`[[[1 5]
 [2 6]
 [3 7]
 [4 8]]]`

A new dimension is created by stacking pairs of elements in the last axis

- 同样还有 `x,y=np.dsplit(a,2)`

**Plt size 默认**

(6.4,4.8)640\*480

`fig, axs = plt.subplots(2, 2)`

`plt.tight_layout()`

`plt.show()`

`plt.savefig("lena_figure.png")`

**marker + Line + color**

Here are some specifiers for each part:

| Marker | Meaning       | Color | Meaning | Line | Meaning |
|--------|---------------|-------|---------|------|---------|
| o      | Circle        | r     | Red     | -    | Solid   |
| s      | Square        | g     | Green   | :    | Dotted  |
| +      | Plus          | b     | Blue    | --   | Dashed  |
| ^      | Triangle (up) | k     | Black   |      |         |

`plt.figure(figsize=(8, 3))`  
`plt.subplot(1, 2, 1)`  
`plt.boxplot(std_scaled_X)`  
`plt.title("Standard Scaler")`  
`plt.subplot(1, 2, 2)`  
`sharey=plt.subplot(1, 2, 1)`  
`plt.boxplot(rbt_scaled_X)`  
`plt.title("Robust Scaler")`  
`plt.show()`

Standardization on the left

Robust scaling on the right

Use the same y axis

- **Accuracy** →  $(TP+TN)/(TP+TN+FP+FN)$  | **Precision** →  $TP/(TP+FP)$  | **Recall** →  $TP/(TP+FN)$  | **F1** →  $2 \times (R \times P)/(R+P)$
  - **ROC** `y_scores = model.predict_proba(X_test)` `fpr, tpr, _ = roc_curve(y_test, y_scores[:, 0], pos_label=0)`
  - **PCA** `scalar = StandardScaler()` `X = scalar.fit_transform(X)` `pca = PCA().fit(X)`
  - Pandas 几乎所有多值选择用[] 列名 / 索引 / 值的清单, 映射关系用{} `map({1: "A"})`
  - `df.loc[0:2]` 左右都闭! (因为这里是人为索引) | `df.iloc[0:2]` 是通常的左闭右开! (系统索引) 是属性不是函数!
  - `df["petal_category"] = df["petal_area"].apply(lambda x: "Large" if x >= 5 else "Normal")`
  - `df.drop(index=.../columns=...)` 简化可不必加 `axis=0` `df.sort_values("sepal_length").reset_index(drop=True)`
  - `pd.concat([names, prices], axis=0, ignore_index=True)` `inplace=True` 在 `df.drop()` 等中改变原 df
  - `covid.isnull().sum().sort_values(ascending=False)` `df.dropna(subset=)` `df.fillna(0)` `df.fill()` `df.bfill()`
  - `X = apple_orange[["Weight", "Size"]].to_numpy()` `y = np.where(apple_orange["Class"]=="apple", 1, 0)`
- `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)` `scaler = StandardScaler()`  
`X_train_std = scaler.fit_transform(X_train)` `perceptron = Perceptron(0.01, 10)` `coef, intercept =`  
`perceptron.fit(X_train_std, y_train)` `X_test_std = scaler.transform(X_test)` `y_pred = perceptron.predict(X_test_std)`

- **class Perceptron:** def \_\_init\_\_(self, lr, n\_iter): self.lr = lr self.n\_iter = n\_iter  
\_decision\_function(...) \_activation\_function(...) fit(...) predict(...)
- **class LogisticRegression(Perceptron):** def \_activation\_function(self, y): return 1.0 / (1.0 + np.exp(-y))  
def predict(self, X): p = self.\_activation\_function(raw\_output) return np.where(p >= 0.5, 1, 0)  
model.predict(X) p = self.\_activation\_function(raw\_output) p = p[:, np.newaxis] return np.hstack((1 - p, p))  
def \_loss\_function(self, y, p): eps = 1e-15 p = np.clip(p, eps, 1 - eps)  
return -np.mean(y \* np.log(p) + (1 - y) \* np.log(1 - p))  
def \_update\_weights(self, X, y, p): error = p - y self.w -= self.lr \* np.dot(X.T, error) / X.shape[0]  
self.b -= self.lr \* np.mean(error)

```

for epoch in range(1, self.n_iter + 1):
    raw_output = \
        self._decision_function(X, self.w, self.b)
    p = self._activation_function(raw_output)
    self._update_weights(X, y, p)
    self.loss.append(self._loss_function(y, p))

```

Predicted probability  
Update weights and bias  
Accumulate the loss

**For loss function:**

```

match self_penalty:
    case "l1":
        penalty = self.lambda * \
            np.sum(np.abs(self.w))
    case "l2":
        penalty = self.lambda * \
            np.sum(self.w ** 2)
    case _:
        penalty = 0

```

**For weights update:**

```

match self_penalty:
    case "l1":
        penalty = self.lambda * \
            np.sign(self.w)
    case "l2":
        penalty = self.lambda * \
            2 * self.w
    case _:
        penalty = 0

```

$\frac{\partial \lambda \sum |w_i|}{\partial w}$   
 $\frac{\partial \lambda \sum w_i^2}{\partial w}$

| Model               | Linear?          | Probabilistic? | Handles Nonlinear? | Major Hyperparameters                          |
|---------------------|------------------|----------------|--------------------|------------------------------------------------|
| Logistic Regression | Yes              | Yes            | No                 | max_iter, C, penalty                           |
| KNN                 | No               | Yes            | Yes                | n_neighbors, weights, metric                   |
| SVM                 | Kernel-dependent | No             | Yes                | C, kernel, gamma                               |
| Decision Tree       | No               | Yes            | Yes                | max_depth, min_samples_split, min_samples_leaf |
| Random Forest       | No               | Yes            | Yes                | n_estimators, max_depth                        |
| Gradient Boosting   | No               | Yes            | Yes                | n_estimators, max_depth, learning_rate         |
| Naive Bayes         | No               | Yes            | No                 | var_smoothing                                  |

- torch.tensor([[[[1, 2]]], [[[3, 4]]]]).squeeze() || unsqueeze(dim=0) like np.newaxis

#### Summary of Tensors and NumPy Arrays

| Features         | PyTorch Tensors             | NumPy Arrays                      |
|------------------|-----------------------------|-----------------------------------|
| GPU support      | Yes                         | No                                |
| Autograd support | Yes                         | No                                |
| Broadcasting     | Yes                         | Yes                               |
| Data types       | Explicit control            | Explicit control                  |
| Use case         | Optimized for deep learning | Towards general computation       |
| Conversion       | To NumPy array: t.numpy()   | From NumPy array: torch.tensor(n) |

```

for epoch in range(1000):
    z = torch.matmul(X, w) + b
    p = torch.clip(1 / (1 + torch.exp(-z)), 1e-6, 1 - 1e-6)
    loss = -torch.mean(y * torch.log(p) + (1 - y) * torch.log(1 - p))

    w.grad = None
    b.grad = None
    loss.backward()
    with torch.no_grad():
        w = 0.01 * w.grad
        b = 0.01 * b.grad

```

Clip to prevent log(0)  
Reset the gradients  
Update the weights and bias

- x = torch.tensor([5.0], requires\_grad=True) → y.backward() → print(w/b.grad)
- z = torch.matmul(x, w) + b || p = 1 / (1 + torch.exp(-z)) || loss = -(y \* torch.log(p) + (1 - y) \* torch.log(1 - p))
- loss.backward() || X\_iris = torch.tensor(iris.data, dtype=torch.float) || ① Layers: nn.Linear and nn.Conv2d || Activations: nn.Sigmoid and nn.ReLU || Loss functions: nn.BCELoss and nn.MSELoss

```

class LogisticRegression(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.linear = nn.Linear(input_dim, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        return self.sigmoid(self.linear(x))

```

The module with four features can be created like this:  
model = LogisticRegression(4)  
To use the logistic regression module in a training loop, we also need:  
An additional module for the loss function: loss\_fn = nn.BCELoss()  
An optimizer for gradient descent: optimizer = optim.SGD(model.parameters(), lr=0.01)

```

for epoch in range(1000):
    for X_batch, y_batch in loader:
        p = model.forward(X_batch)
        y_batch = y_batch.float().unsqueeze(1)
        loss = loss_fn.forward(p, y_batch)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

```

Log loss (binary cross entropy)  
optimizer.zero\_grad()  
loss.backward()  
optimizer.step()

- train\_data = datasets.MNIST(root="./data", transform=transforms.ToTensor(), train=True, download=True)  
np\_image = tensor\_image.numpy() np\_image = np\_image.transpose(1, 2, 0) plt.imshow(np\_image) plt.show()

```

class MnistCnnWithDropout(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(2)
        self.flatten = nn.Flatten()
        self.linear1 = nn.Linear(64 * 7 * 7, 128)
        self.dropout = nn.Dropout(0.5)
        self.relu3 = nn.ReLU()
        self.linear2 = nn.Linear(128, 10)

```

The hidden layer  
The activation function for the hidden layer  
The output layer

```

class RNNModel(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.rnn = nn.RNN(input_dim, hidden_dim, batch_first=True)
        self.linear = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        h_n = self.rnn(x)
        return self.linear(h_n[-1])

```

The first output is not used